

Mesh Dependence in PDE-Constrained Optimisation with Firedrake, pyadjoint, and PETSc TAO

Reproducing Schwedes’ experiment and extending it to the inner Krylov solve

Bert Jim Chang
Supervisor: David A. Ham
Department of Mathematics

Context

PDE-constrained optimisation is the problem of choosing an input to a physical system, described by a partial differential equation, so that its solution matches some target as closely as possible. This can be used in a wide range of applications, for example:

- ▶ Reconstructing the source of a chemical spill from downwind measurements
- ▶ Designing aeroplane wings that minimise drag
- ▶ Configuring heat sources to reach a target temperature distribution across a domain.

In each case the unknown is a *function* on a domain, not a vector of numbers.

Firedrake [1] is an open-source Python library for solving PDEs by the finite element method, and pyadjoint [2] is its adjoint extension. Together they let a user write these problems in code that closely mirrors the mathematical formulation; the optimisers are provided by PETSc TAO. There is a catch: the natural way to measure the size of a function is a Hilbert-space inner product, not the standard Euclidean one. Using the wrong choice causes the number of optimisation iterations to grow polynomially as the mesh becomes finer — precisely the meshes users need for accurate results.

The finite element method

Computers cannot store or manipulate functions exactly — they only work with finite lists of numbers. The **finite element method** (FEM) bridges this gap by dividing the domain into small polygons, called a *mesh*, and approximating the unknown function as a low-degree polynomial on each cell. Finding the function then reduces to finding a finite list of polynomial coefficients, which a computer can handle.

FEM is the standard tool for numerical PDE simulation because it (i) works on domains of arbitrary shape, including curves and holes, and (ii) lets the user place small cells wherever the solution varies rapidly, and larger cells elsewhere, so that computational effort is spent only where accuracy is needed. As a result, real-world meshes are almost never uniform.

The mesh-dependence phenomenon

Every gradient-based optimiser needs to measure the size of gradients: how big a step to take, and how close it is to converging. It does this using an *inner product*. Standard optimisers use the Euclidean inner product, treating each basis-function coefficient as a plain number, without accounting for the underlying mesh.

Schwedes et al. [3] show that on non-uniform meshes this is exactly wrong. Different cells have different sizes, and ignoring this makes iteration counts grow polynomially with the ratio $h_{\max}/h_{\min}$ (the largest cell size divided by the smallest). An optimiser that uses the natural L^2 inner product on the function space — which does account for cell sizes — is mesh-independent.

The fix is the *Riesz map* $\mathcal{R}$: a linear operator that converts a raw gradient into a properly scaled update direction the optimiser can use. In the discrete L^2 setting,

$$\mathcal{R}_{L^2} = M_h^{-1},$$

the inverse of the *mass matrix* M_h , whose entries $(M_h)_{ij} = \int \phi_i \phi_j$ encode L^2 inner products between the finite-element basis functions ϕ_i . Multiplying a gradient by M_h^{-1} cancels the cell-size scaling.

Test problem

A *test problem* is a benchmark used to check whether a numerical method works and to compare methods to each other. We use the classic *Poisson optimal control problem*: simple enough that the maths is transparent, but rich enough to display the mesh-dependence pathology.

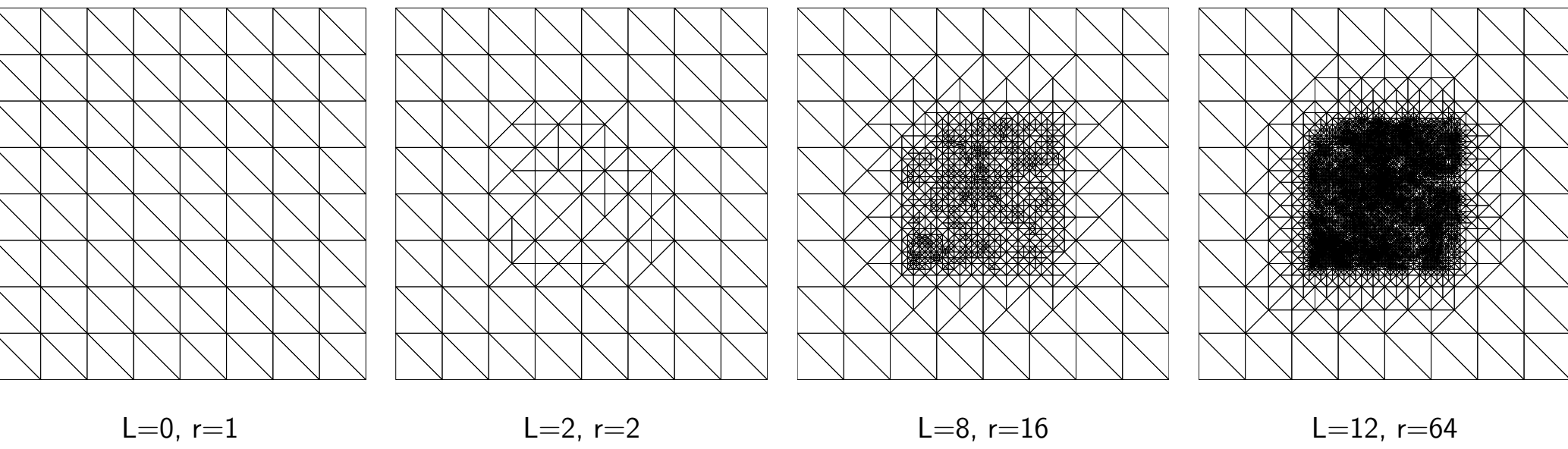
On the unit square $\Omega = (0, 1)^2$, we look for a source term m so that the state u matches a target d as closely as possible:

$$\tilde{J}(m) = \frac{1}{2} \|u - d\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m\|_{L^2(\Omega)}^2, \quad -\Delta u = m, \quad u = 0 \text{ on } \partial\Omega,$$

with $d = \sin(\pi x) \sin(\pi y)$ and regularisation $\alpha = 10^{-4}$. The gradient of the reduced functional $\tilde{J}$ is computed automatically by pyadjoint.

Meshes

We adapt the method of [3, Sect. 2.3.2.1] to produce a hierarchy of increasingly non-uniform meshes. Starting from a uniform triangulation, at each level a random subset of cells inside an interior region is refined. The largest cells stay essentially the same size; the smallest cells shrink geometrically, so $h_{\max}/h_{\min}$ doubles every two levels.



Objectives

1. **Check whether Schwedes’ problem is still present** in the modern Firedrake + pyadjoint + PETSc TAO stack, ten years on. Reproduce their experiment and identify which TAO solvers work correctly and which do not.
2. **Fix the mesh-dependence in TAO/NLS** using a Riesz-map preconditioner on the inner CG. Schwedes’ experiment does not cover this level of the solver, so this is a novel contribution.
3. **Deliver the fix as a pyadjoint feature.** Users should get correct mesh-independent behaviour from a single options entry, without needing to know about Riesz maps.

Reproducing the experiment (outer L-BFGS)

L-BFGS is a widely-used quasi-Newton optimiser: it builds up a low-rank approximation of the Hessian across iterations and updates the solution using this approximation. Two L-BFGS optimisers run on the same problem, on the same mesh sequence, to the same tolerance. The only difference is the inner product each uses internally.

Inner product used	L=0	L=2	L=4	L=6	L=8	L=10	L=12	L=14
Realised $h_{\max}/h_{\min}$	1	2	4	8	16	32	64	128
Euclidean ℓ^2	13	39	71	104	205	401	701	1269
Function-space L^2	6	8	6	8	7	8	8	6

The ℓ^2 row grows nearly $100\times$ across the sweep, matching the polynomial dependence predicted by Schwedes. The L^2 row stays bounded between 6 and 8. TAO’s `lmvm` solver reproduces the flat row once its default limited-memory history is raised from five to one hundred.

Preconditioning

Some iterative solvers – like *conjugate gradient* (CG) – can converge very slowly when the underlying matrix is badly conditioned. A *preconditioner* P transforms a difficult linear system $Ax = b$ into an easier one,

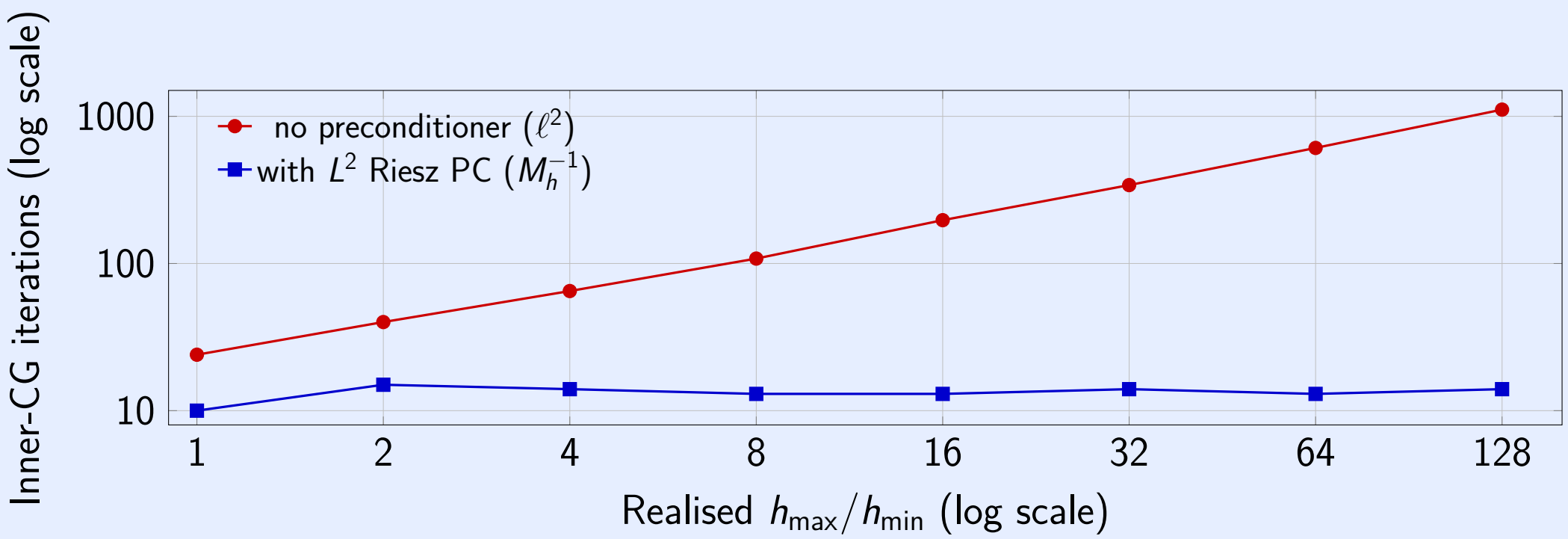
$$P^{-1}Ax = P^{-1}b,$$

whose matrix $P^{-1}A$ is close to the identity. The iterative method then converges much faster — often in a number of iterations independent of problem size.

For the reduced Hessian on non-uniform meshes, the Riesz map M_h^{-1} is exactly the right preconditioner: it cancels the cell-size scaling that would otherwise ruin the conditioning.

New result: preconditioning the inner CG of TAO/NLS

TAO offers many optimisers. L-BFGS (previous section) builds a Hessian approximation across iterations, so a correct choice of inner product is enough to make it mesh-independent; it does not need a preconditioner. **NLS** (Newton Line Search) is Newton’s method: at each outer step it solves the exact reduced-Hessian system for a search direction, using inner CG. On our test problem NLS takes only two outer iterations, but each inner CG grows with mesh non-uniformity unless preconditioned. The discrete Riesz map M_h^{-1} removes this growth completely.



The uncorrected inner solve inherits the same mesh-scaling pathology as the outer optimiser: iteration counts grow nearly $50\times$ across the mesh sweep. The L^2 Riesz-map preconditioner does at the Krylov level what changing the inner product does at the optimiser level, restoring mesh-independence completely. This connection is pointed at by [3, Sect. 1.3.3] via [4], but is not tested experimentally in the book.

Work plan

- ▶ **Ship the fix as a pyadjoint feature.** Users should be able to enable the Riesz preconditioner with a single options entry; the inner product should be read off the Control (single source of truth). This is the main remaining deliverable.
- ▶ **Extend to H^1 and other Hilbert spaces on the control**, and characterise when each Riesz map is the correct preconditioner.
- ▶ **Continue the bqn1s diagnosis.** At PETSc defaults it is mesh-dependent because it never receives the M_h initial-Hessian seed on its LMVM matrix.

References

[1] Rathgeber et al. (2016). Firedrake: automating the finite element method by composing abstractions. *ACM TOMS* 43(3).
[2] Mitusch, Funke, Dokken (2019). dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *JOSS* 4(38).
[3] Schwedes, Ham, Funke, Piggott (2017). *Mesh Dependence in PDE-Constrained Optimisation*. SpringerBriefs.
[4] Kirby (2010). From functional analysis to iterative methods. *SIAM Review* 52(2).